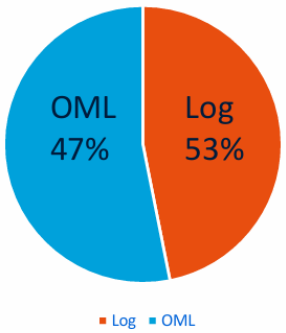
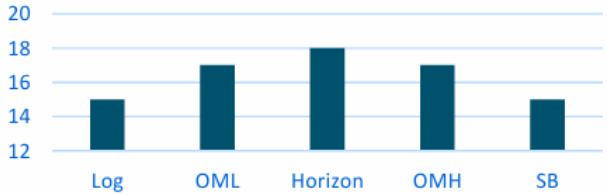
2 – Advanced Visualizations

nD data, Overplotting reduction, (Geo)spatial data, Time data, Graph data

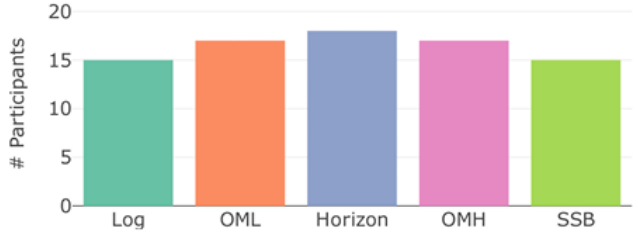
1D Visualization

- 1D Data → Each item has 1 attribute, map to the best visual variable
- Take care of:
  - Starting at the base value
  - Including axis descriptions
  - Descriptive titles / captions
  - Sensible tick marks and axis lines
  - Use color to reference elsewhere, or encode additional information
- Use pie charts only for 2 values



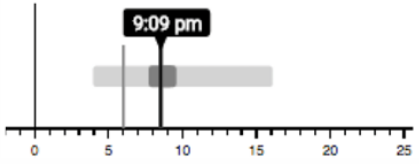


Default excel bar-chart

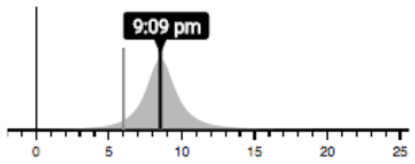


(a) Number of participants per design.

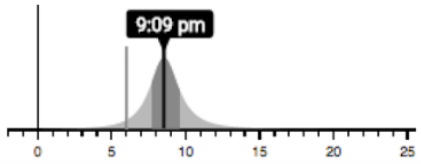
Interval plot (box plot)



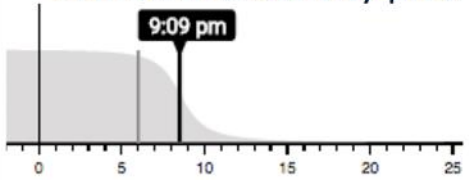
Probability density plot



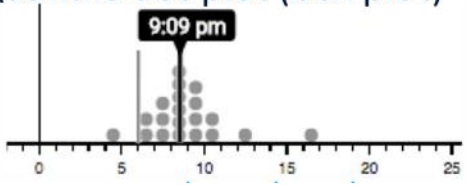
Probability density + interval plot



Cumulative density plot

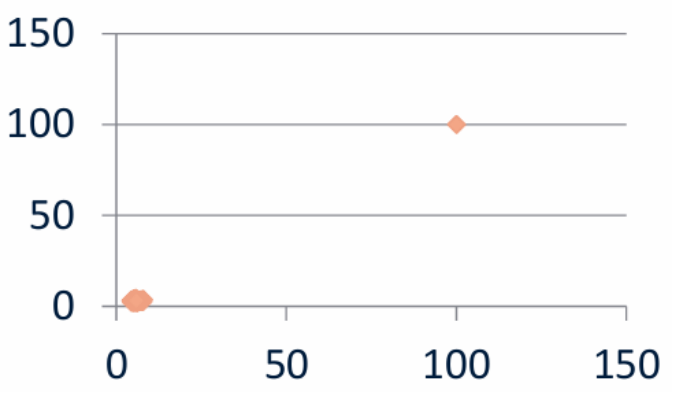


Quantile dot plot (box plot)

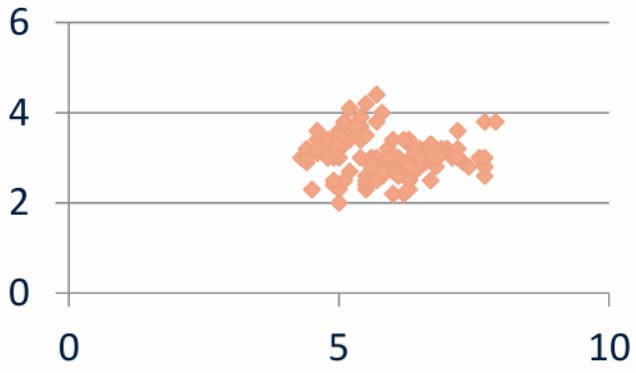


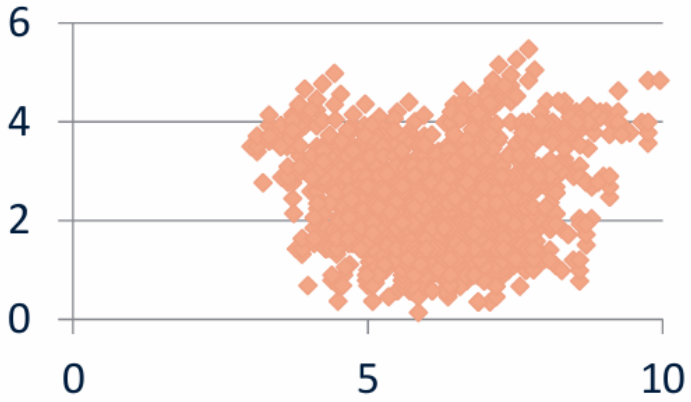
2D Visualization

- 2D Data → Each item has 2 attributes, map to the best visual variable
- Most common form: **Scatter Plot**
- Too much data can lead to **overplotting**



Outliers can hurt the scaling

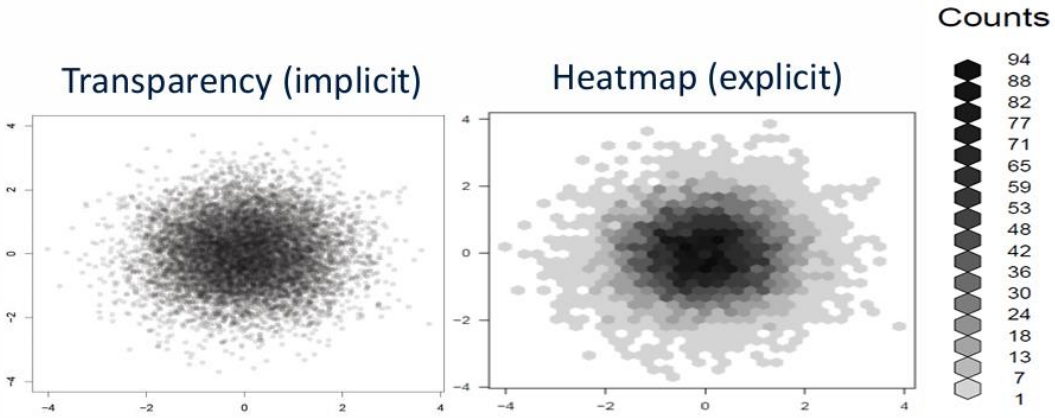




Too much data looks cluttered

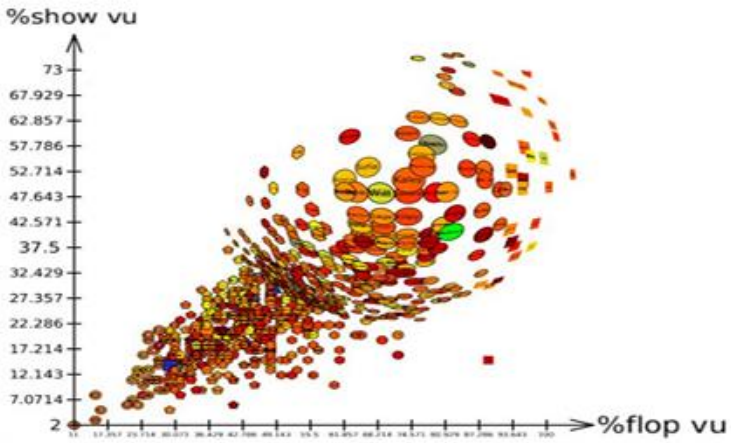
Overplotting Reduction

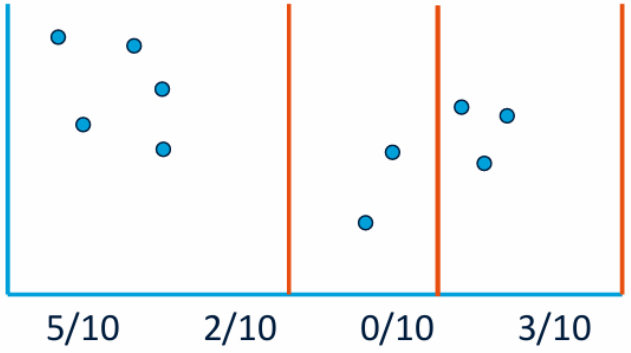
- **Jittering** (add random noise to data points)
  - Data is no longer completely accurate
  - Does not work with too much overlap
- **Encoding** (encode density with saturation)
  - Cannot identify single values
  - Not suitable for small datasets
  - Low-density areas blur together
- **Sampling** (select a subset of items to keep)
  - Data is lost
  - Dependent on sampling strategy

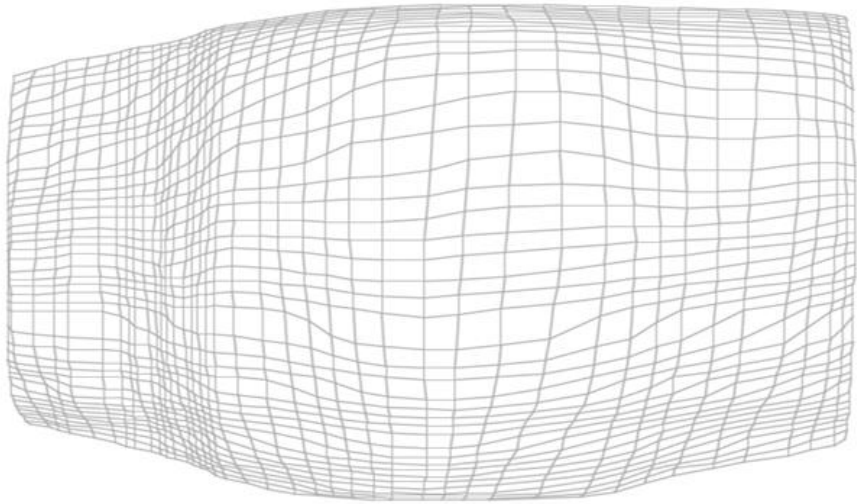


Overplotting Reduction

- **Local Distortion** (fish-eye view on hover)
  - Interactivity required
  - Visual mapping obscured when distorting
- **Global Distortion** (distort grid according to density)
  - Visual mapping to position is always obscured
  - Does not work great for non-uniform distributions

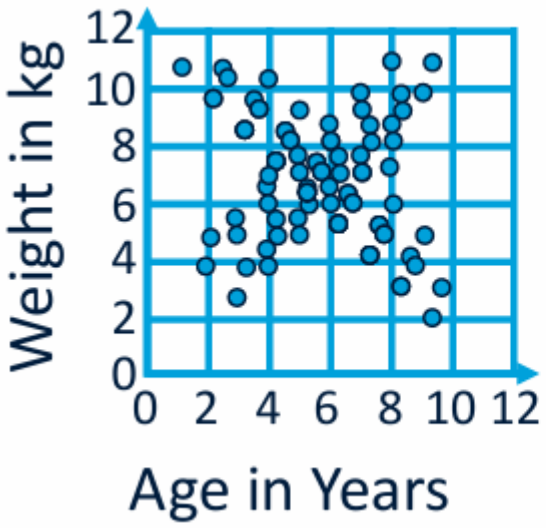


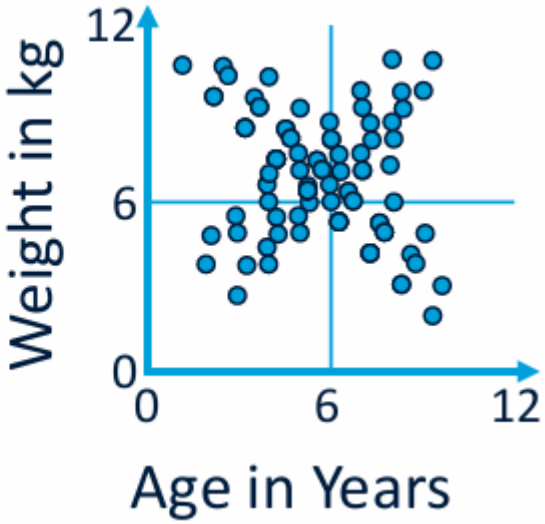




3D Visualization

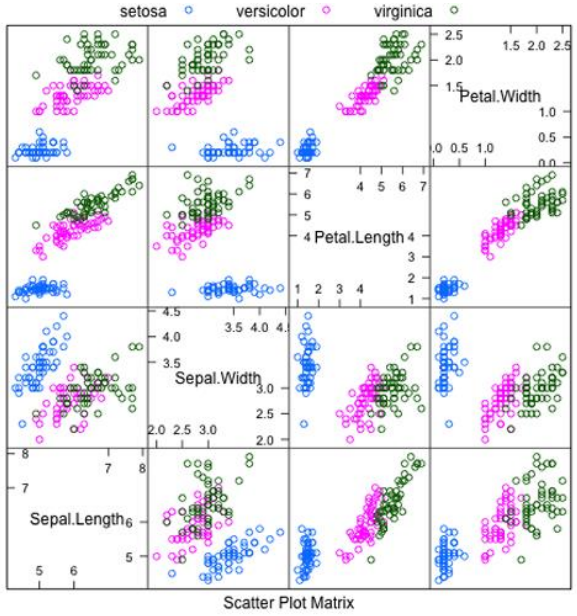
- 2D Scatterplot + Hue (categorical) or Area (quantitative / ordinal)
- 3D Scatterplot is usually avoided unless data is spatial
- Use as few gridlines as needed

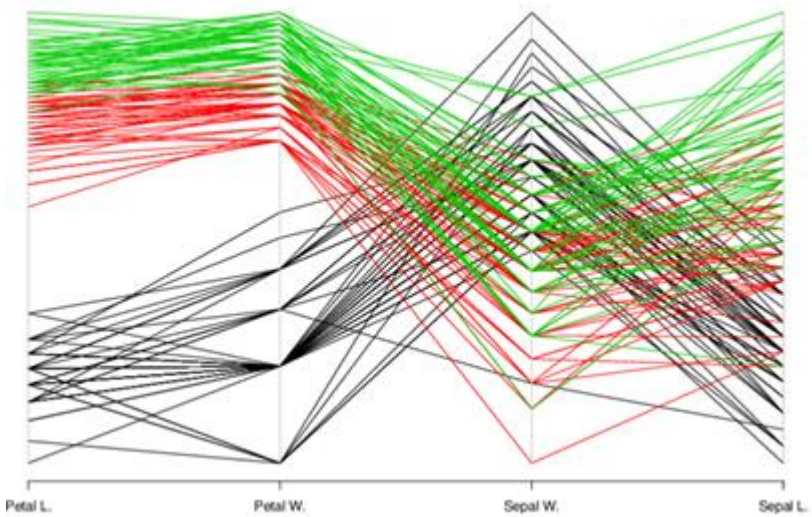




nD Visualization

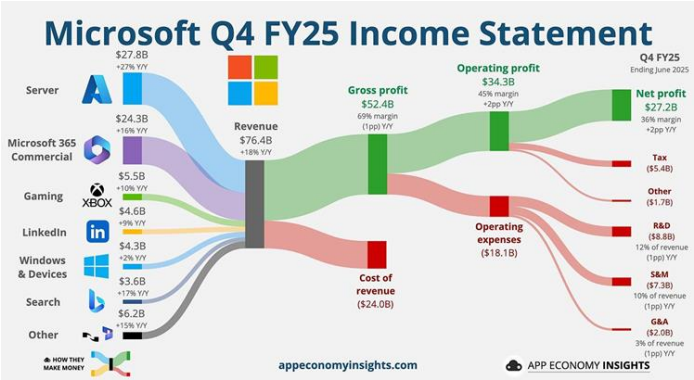
- **Scatterplot-matrix**: one scatterplot for every pair of attributes
  - Great for comparing pairwise trends, difficult to identify items (scales up to n≈10)
- **Parallel Coordinates**: map variables to x-positions, lines to connect items
  - Great for browsing clusters, ordering of variables is crucial

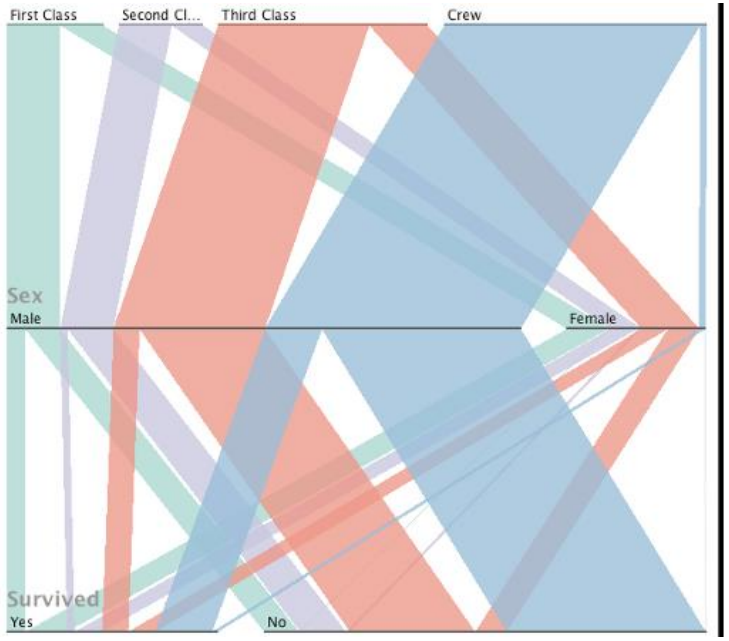


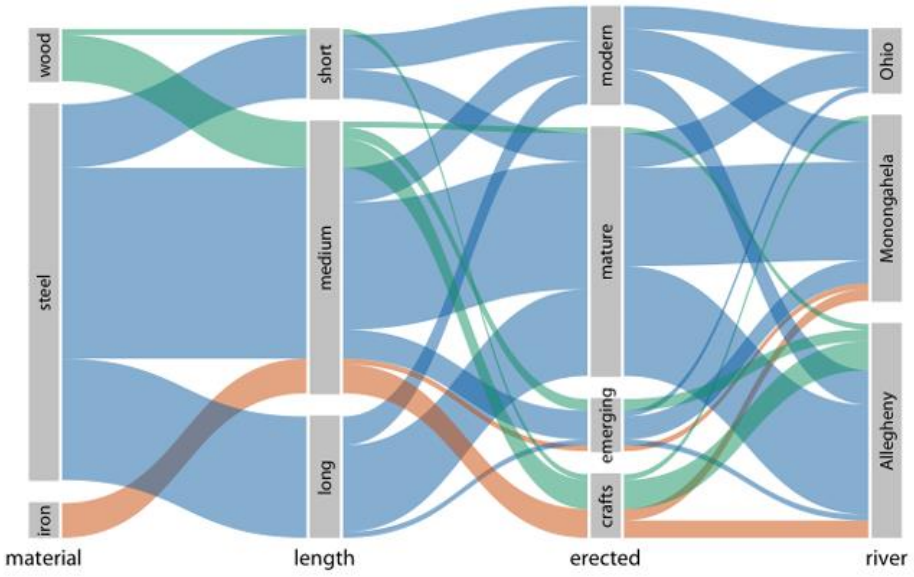


nD Visualization

- **Parallel Sets**: parallel coordinates for categorical data
  - Width of the line is the size of the group
  - Can be mixed with parallel coordinates
  - Closely related: **Sankey Diagram**



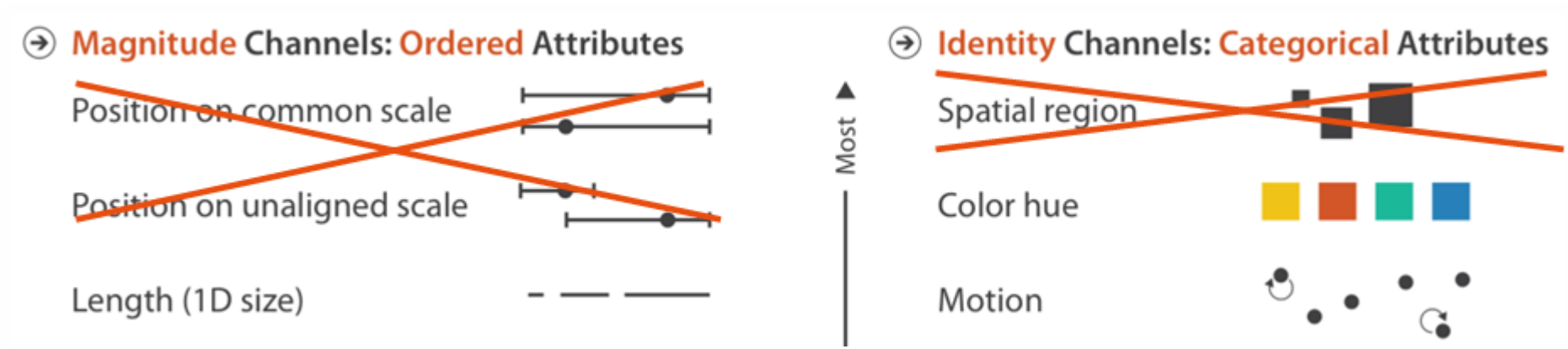




Geospatial Data

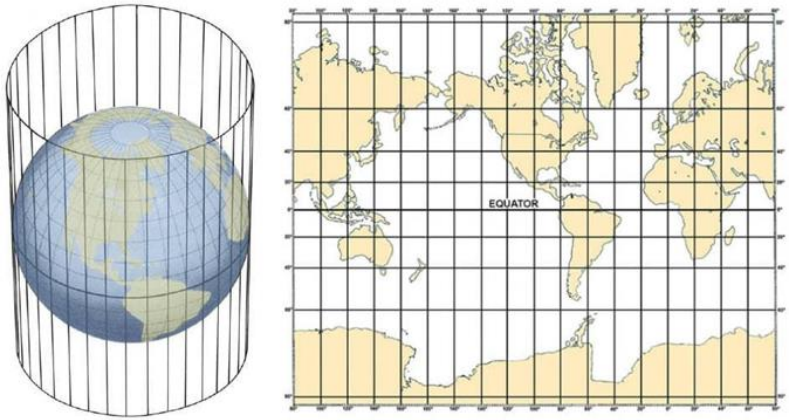
- **Geospatial Data**: data with a (relevant) location
- A location may be **point data** (GPS, address, etc.) or **area data** (postcode, country, region, etc.)
- A location may be **static** (constant) or **dynamic** (changes over time)

- Key foundation of geospatial visualization: we cannot use position!

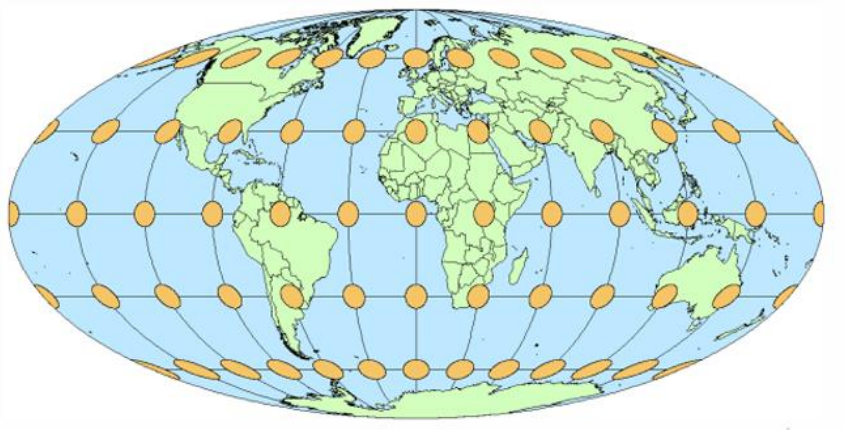


Projections

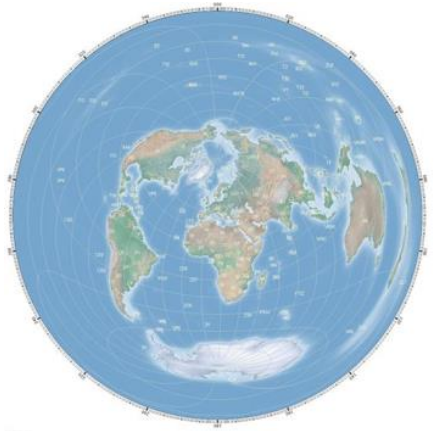
- **Conformal**: directions are correct, areas are distorted
  - Areas are only distorted on a global scale, local shapes are fine (**uniform distortion**)
  - The further from the equator, the more areas are distorted (**non-uniform distortion**)
- **Equal-Area**: areas are correct, directions are distorted
- **Equidistant**: all distances to one (or two) points are correct



Conformal



Equal-Area

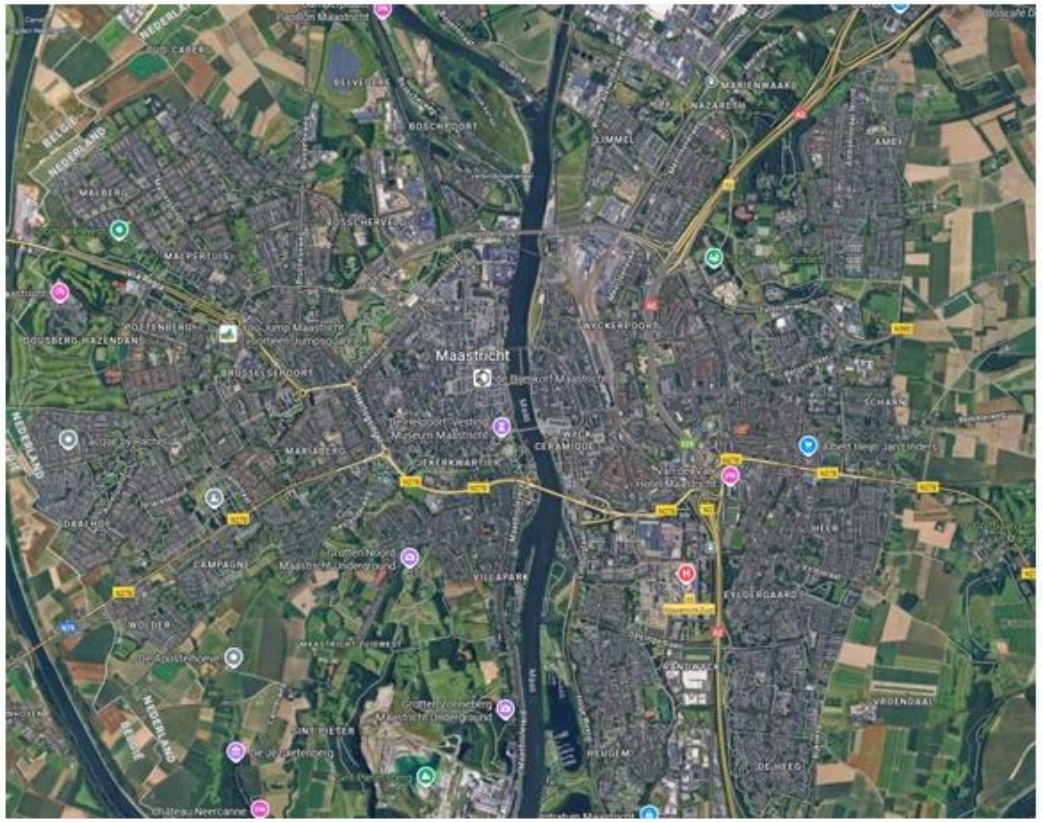


Equidistant

Abstract Maps

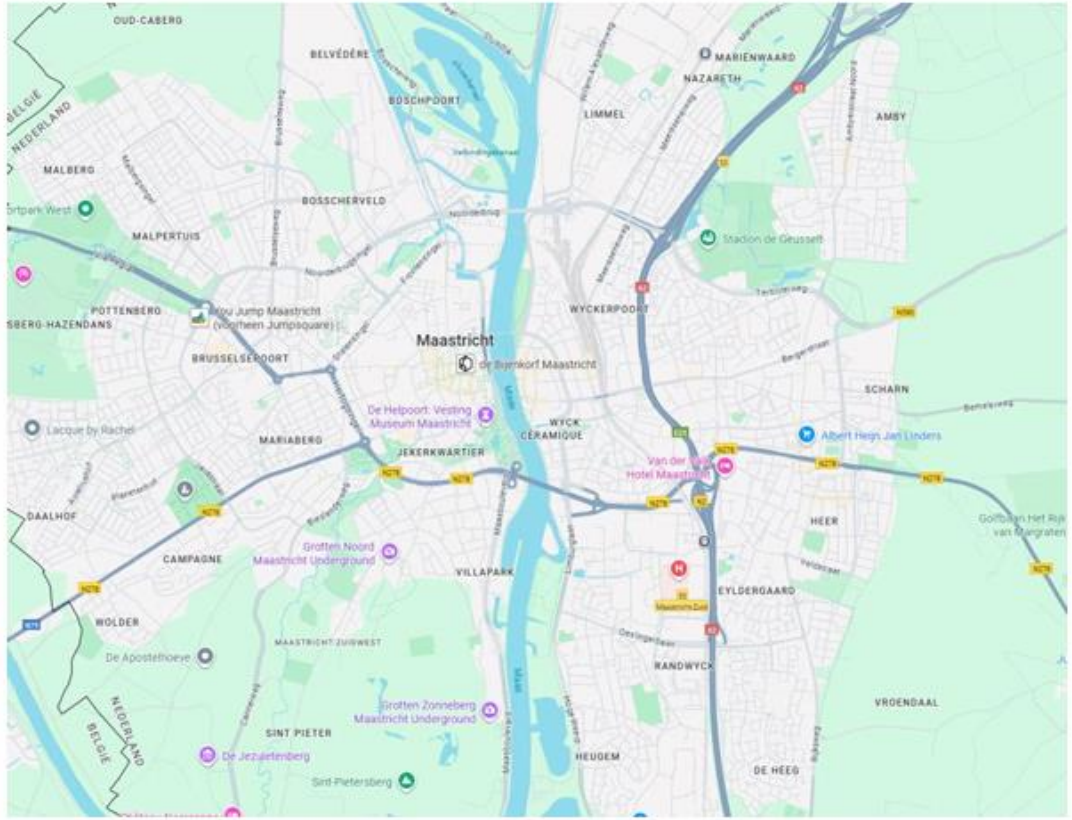
exploration

Raw “data” => Satellite, survey



task specific

Abstract map: Only “important details”



Choropleth Map

- **Choropleth Map**: color geospatial areas by their value

- Familiar
- No overplotting
- Small areas are hard to see
- High values are not highly visible

- Use them if you have (mostly) regular sized areas, or if the value is highly correlated to land area (population is not correlated to area!)

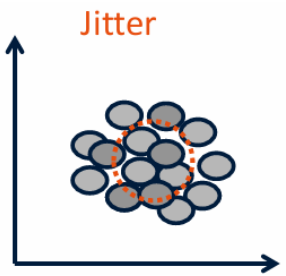
Prism Map

- **Prism Map**: value is given by height (and color)
- Requires **spatial autocorrelation** (areas close in space have similar values)
- Requires small value ranges

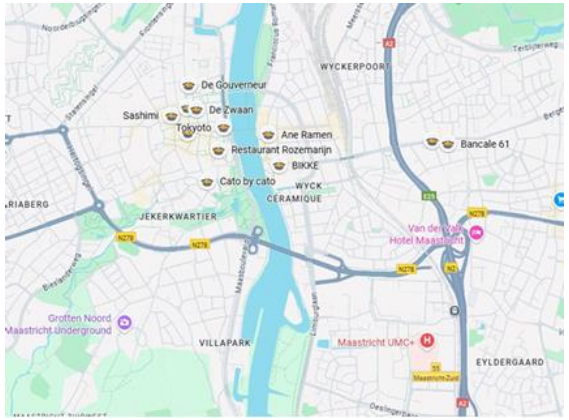
(a) Choropleth (b) Monochrome Prism (c) Coloured Prism

Symbol Map

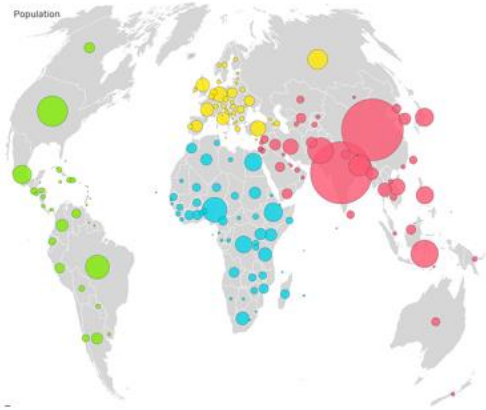
- **Symbol Map**: encode data by drawing marks on the map
- Overplot reduction → Data is spatial, so no jitter!



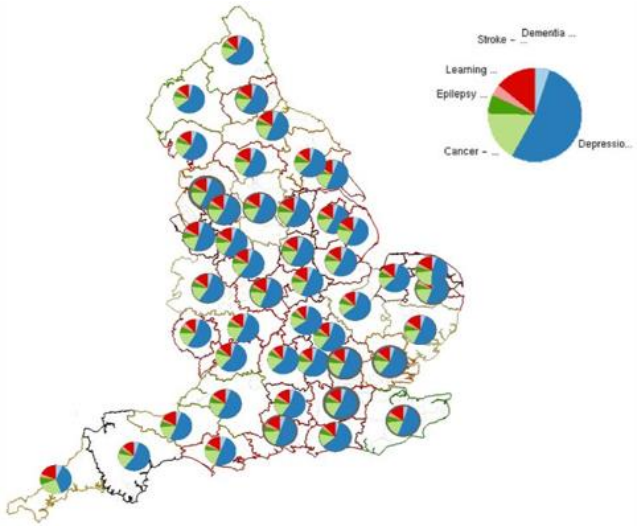
- **Glyphs**: small independent visual objects that represent a data item
- Can use multiple visual variables to encode nD data



Mark: point



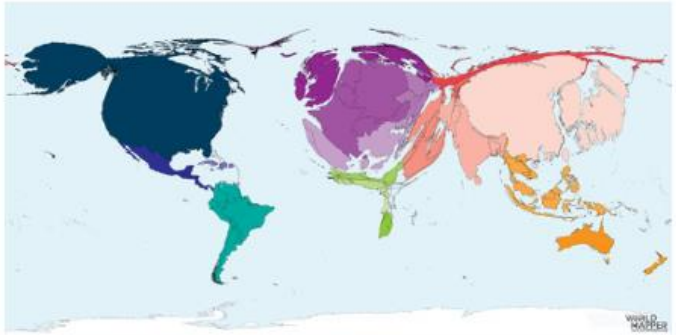
Mark: circle



Mark: glyph

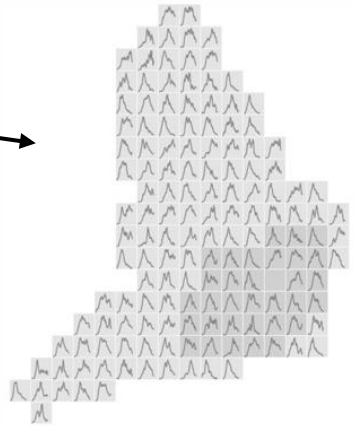
Cartogram

Diffusion Cartogram



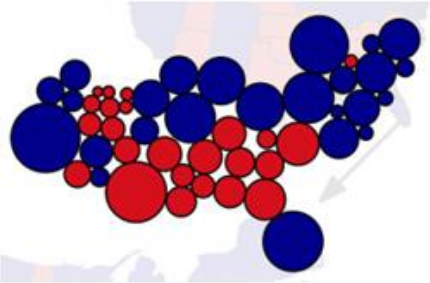
Mosaic Cartogram

- **Cartogram**: value is given by area
  - **Diffusion**: inflate every region to the correct size
  - **Mosaic**: represent all regions using grid cells proportional to the value
  - **Gridmap**: special case of Mosaic cartogram (one cell per region)
  - **Non-Contiguous**: shrink all regions in place
  - **Dorling**: represent all regions as non-overlapping circles
  - **Demer**: represent all regions as non-overlapping squares
  - **Rectangular**: represent all regions as touching rectangles



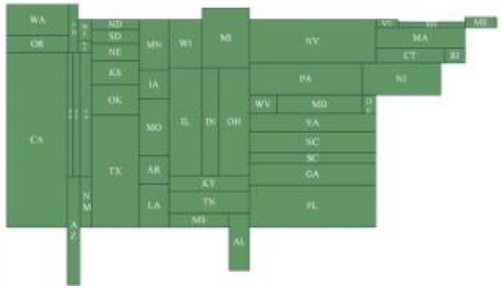
Non-contiguous Cartogram

Dorling Cartogram

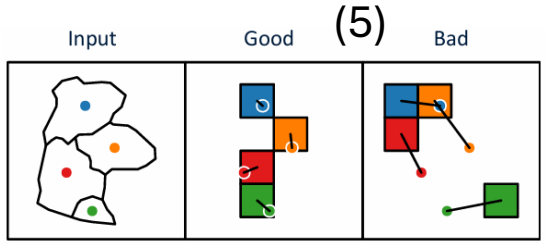


Demer Cartogram

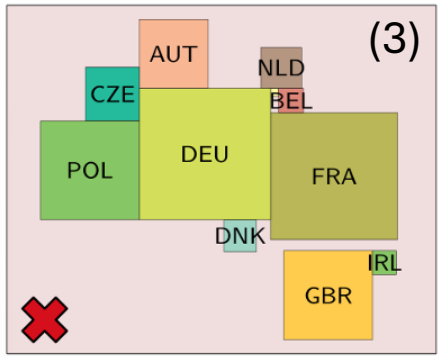
Rectangular Cartogram



Cartogram Quality



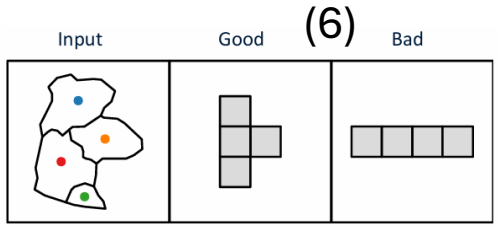
(5)



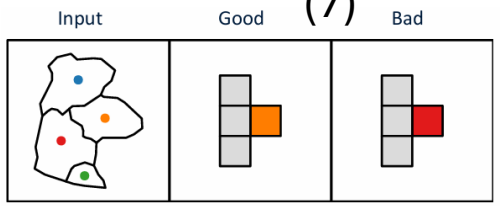
(3)

(2)

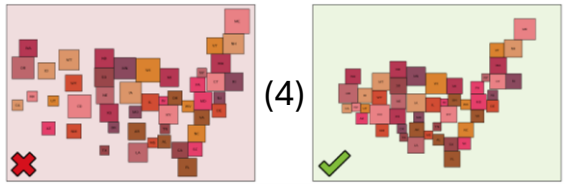
1. **Cartographic error**: region size should match value
2. **Shape deformation**: regions should resemble their geographical shape
3. **Relative directions**: maintain relative directions between regions
4. **Topological accuracy**: geographically-adjacent regions should be adjacent
5. **Spatial deformation**: regions should be placed like their spatial position
6. **Global shape**: the overall shape should match the geographical shape
7. **Local shape**: regions on identifiable positions should match
8. **Complexity**: area should be easy to visually estimate
9. **Data-ink ratio**: should always be high

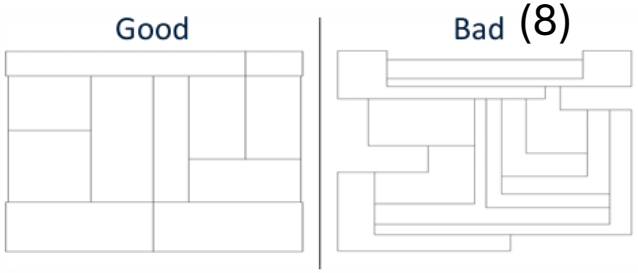


(6)



(7)


(4)

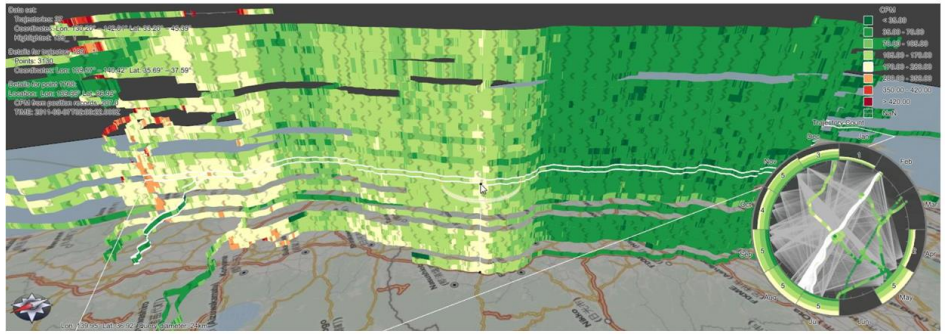


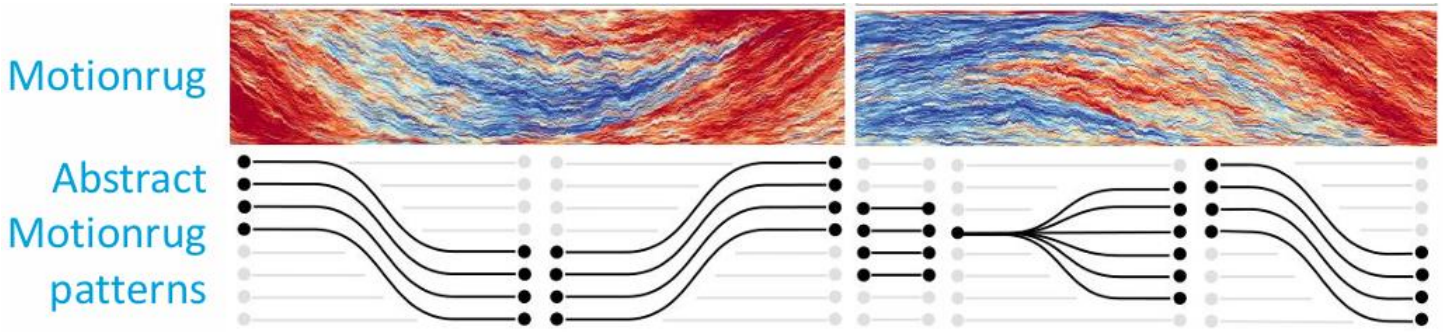
(8)

Trajectories

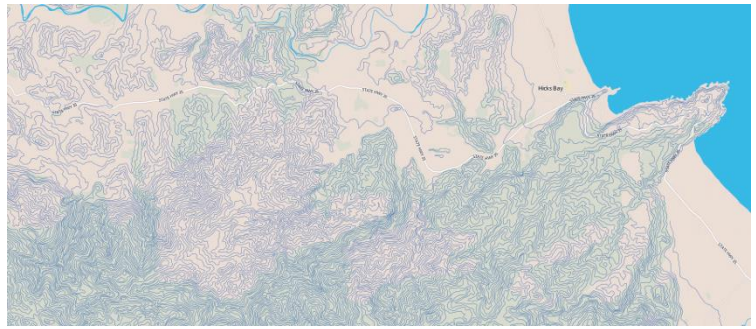
- **Trajectory Data**: geospatial position of an entity over time
- Multiple trajectories very quickly leads to overplotting (no jitter or distortion!)

- **TrajectoryWall**: spacial information in 2D, stack trajectories in 3D
- **Motionrug**: reduce spatial information to 1D y-axis, x-axis is time
- Extra information can be encoded using color (e.g. speed)

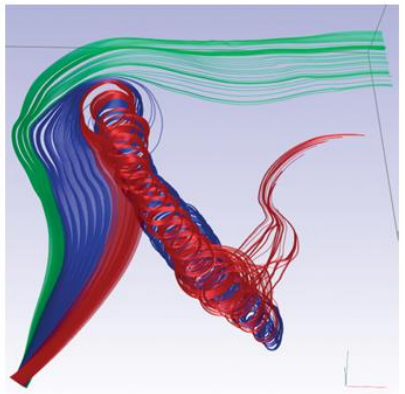




Other Geometry

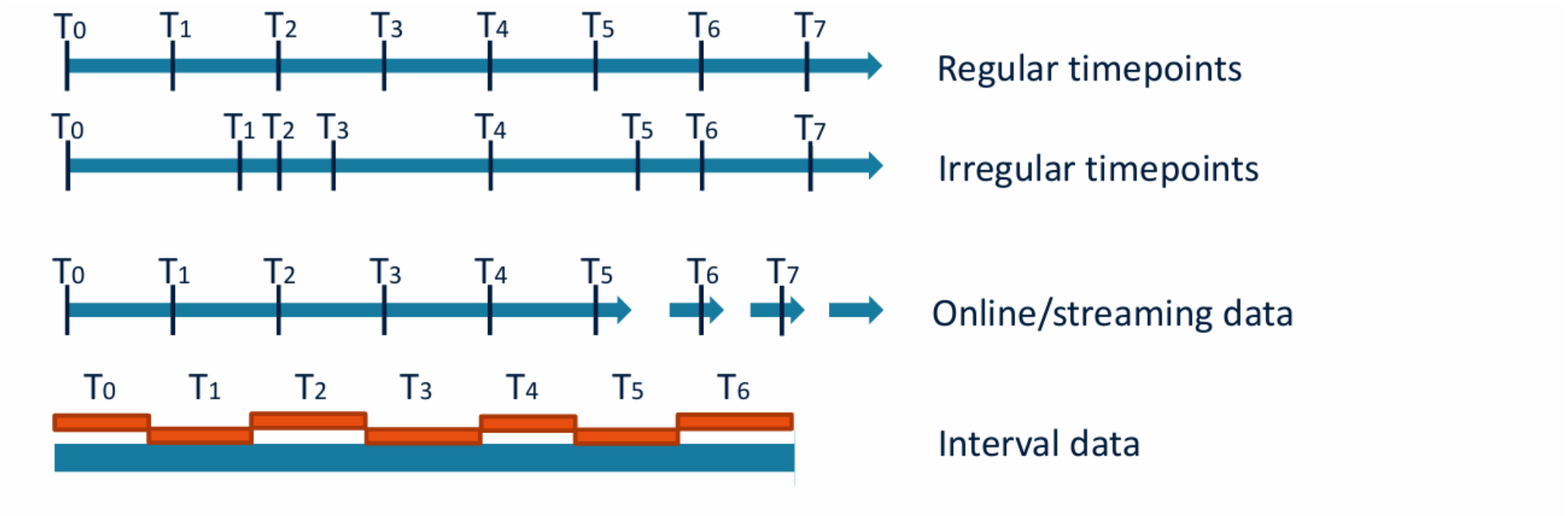


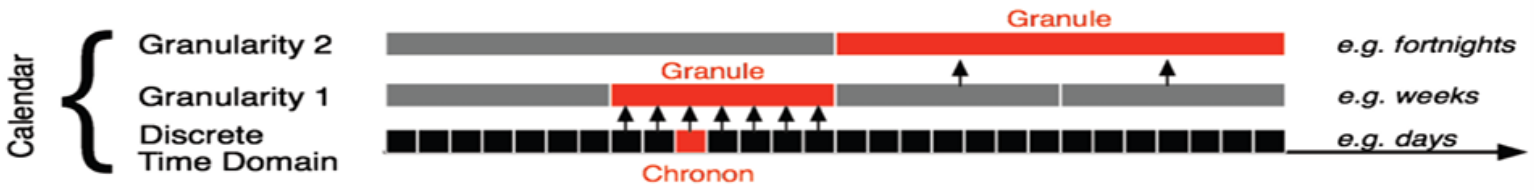
- Not all spatial data is geographic, e.g. 3D scans, medical imaging
- **Scalar Fields**: single value associated with each spatial cell
  - **Slicing**: only show information of 2 dimensions at once
  - **Isocontours**: a set of isolines that represent the contours of a particular scalar level
  - **Direct Volume Rendering**: generate an image from a particular 3D viewpoint
- **Vector Fields**: multiple values associated with each spatial cell
  - **Flow Glyphs**: e.g. show direction of vector using arrow
  - **Geometric Flow**: show derived geometric data based on computed *seed points*
- **Tensor Fields**: many values associated with each spatial cell



Time Data

- **Time Data**: any data with a *time* dimension

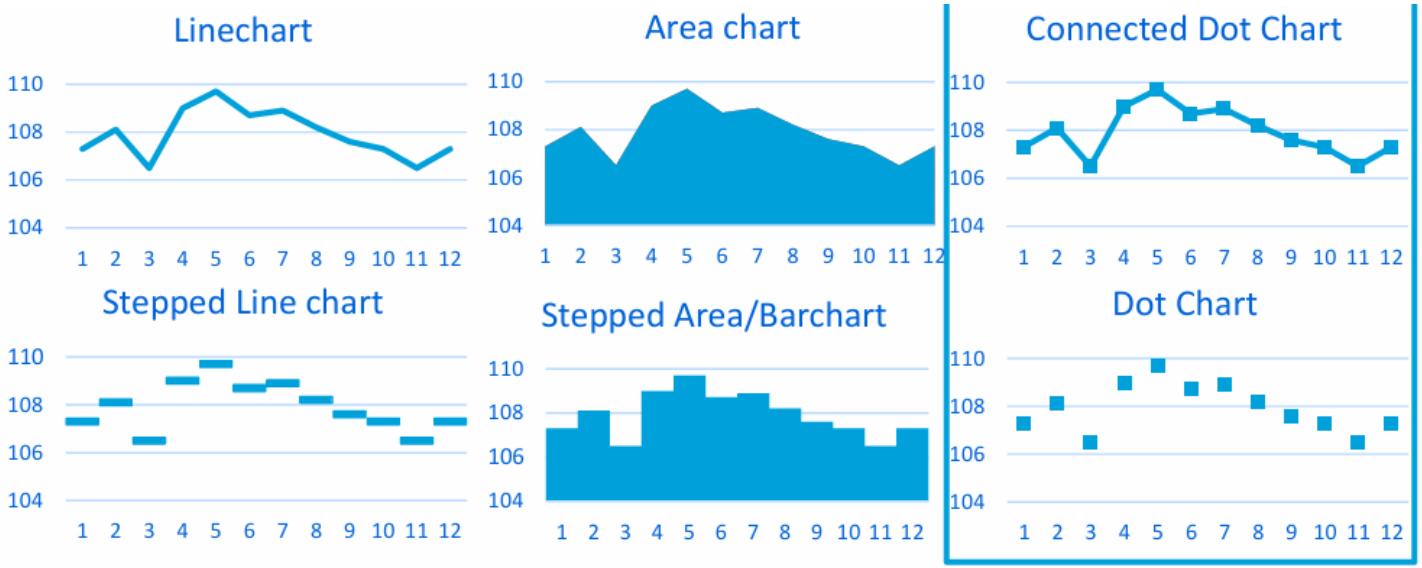




Hierarchical timescheme

Time Data

- **Periodic Repetition**: fixed time period in which there is a repeating pattern
  - E.g. traffic congestion per day / week, temperature per day (not week!)
- **Aperiodic Repetition**: irregular period in which there is a repeating pattern
  - E.g. structure of a song (repeating chorus), national debt (spikes during wars)
- **Linear Time**: no semantically repeating patterns

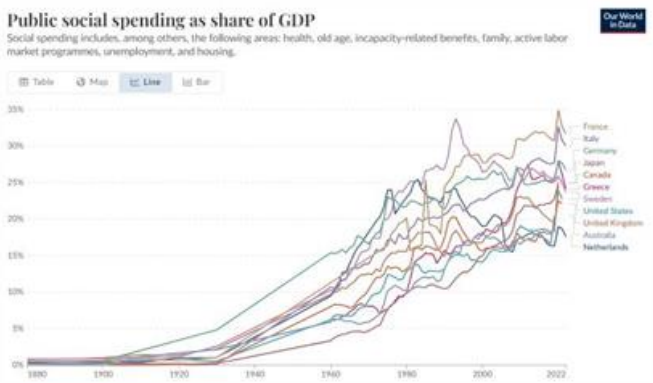


(interpolation is relevant, e.g. time)

(interpolation not relevant)

Multiple Timeseries

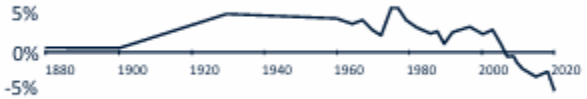
- Assumptions: all timeseries are linear time, sampled at the same time



- **Superimposed**: stack them on top of each other
  - Space efficient, but hard to read individual series

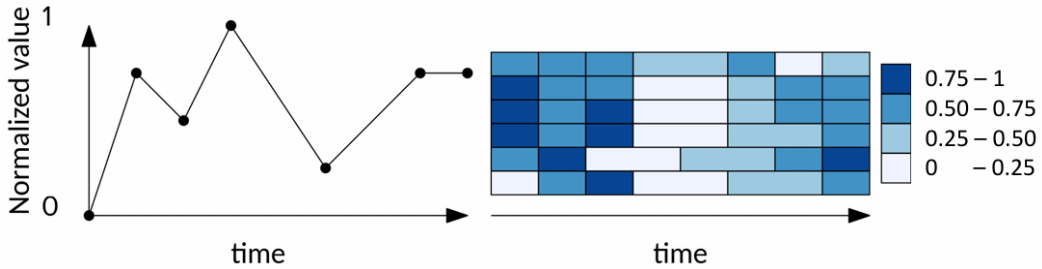
- **Juxtaposed**: show them side-by-side (grid)
  - Individual series readable, but high mental cost for tasks involving multiple series

- **Explicit Encoding**: directly encode the task using visual variables
  - Good for specific tasks, but individual series are hidden (+ higher conceptual separation)

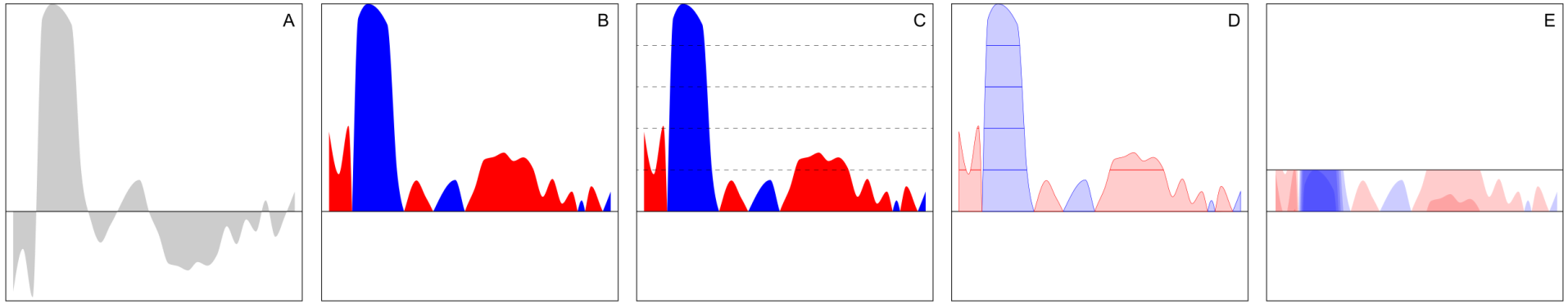


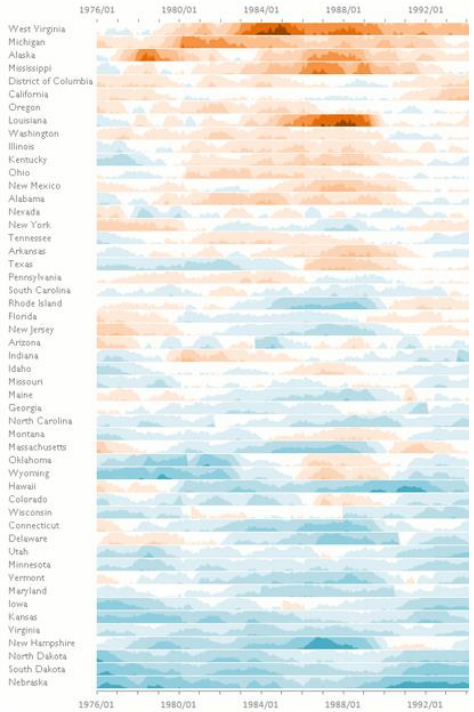
Multiple Timeseries

- Overplotting → No jitter or global distortion!



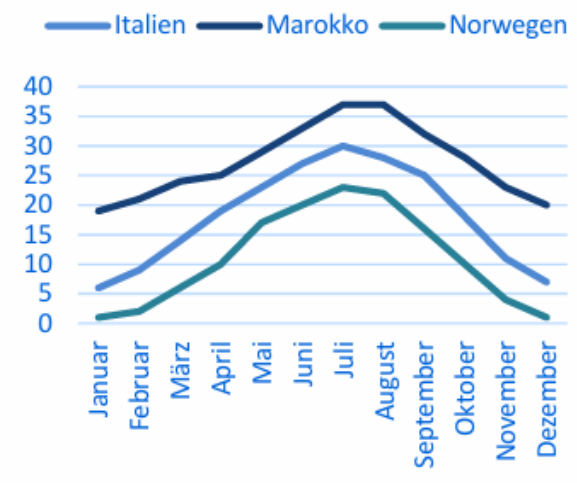
- Juxtaposition can be used if there are not *too* many timeseries
- **Heatmap**: x-axis is time, y-axis is timeseries, saturation is value
- **Horizon Plot**: combination of line chart and heatmap



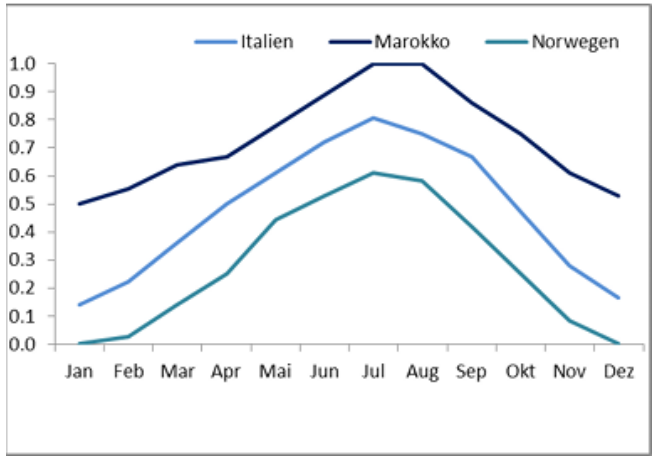


Timeseries Normalization

- No Normalization

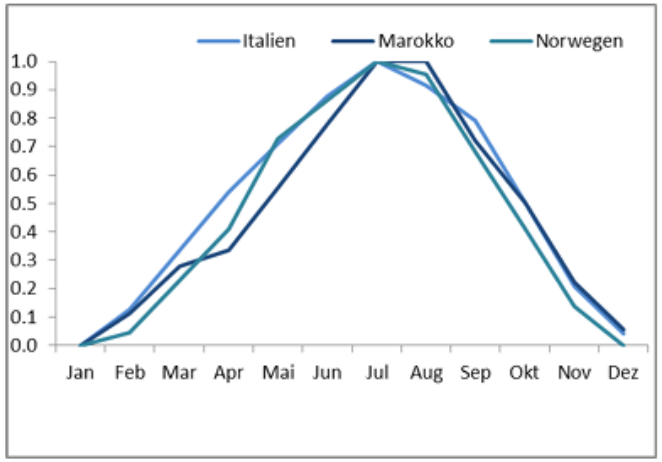


- **Global Normalization**: normalize over *all* timeseries



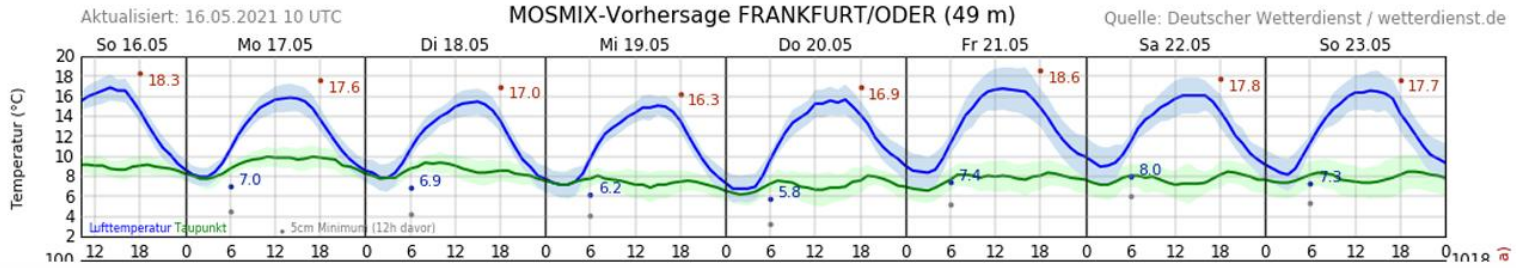
(compare *relative* values)

- **Local Normalization**: normalize *per* timeseries



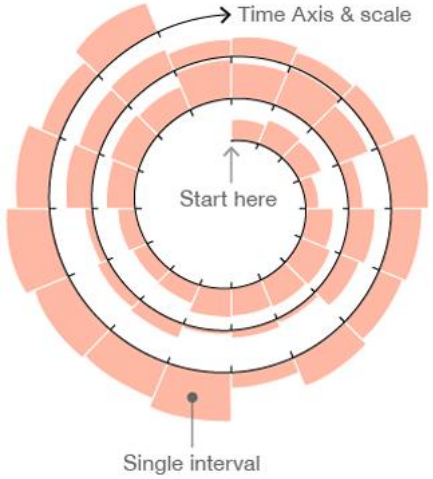
(compare relative *trends*)

Periodic Time

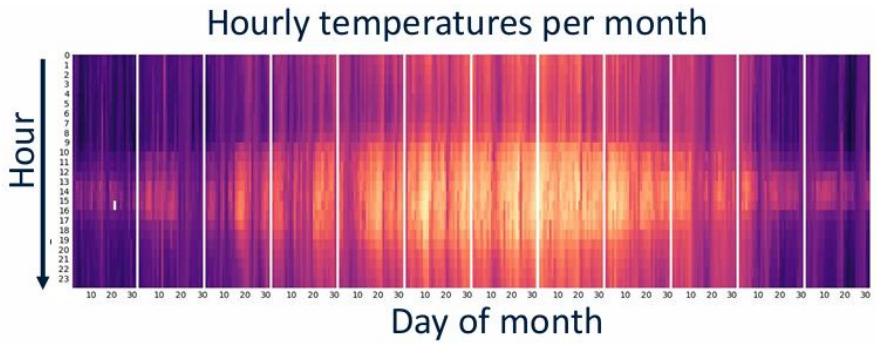


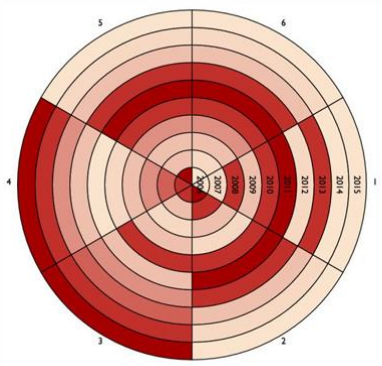
- Tasks: identify periodicity + periodic pattern

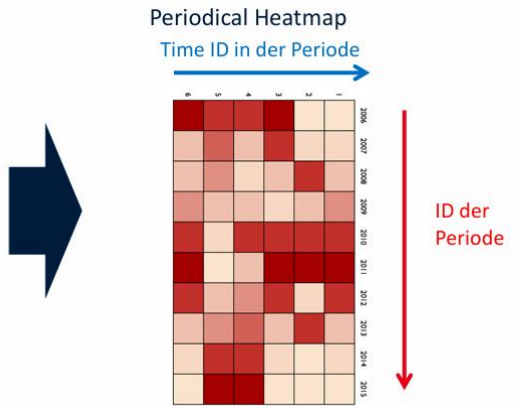
- **Line Chart**: highlight periodicity
- **Spiral Graph**: spiral coordinates
  - Outer rings are larger than the inner rings!
  - Solution: go back to squares (less aesthetic, but more usable)



- **Heatmap**: observe periodicity for multiple timeseries







Aperiodic Time

(arc diagram)

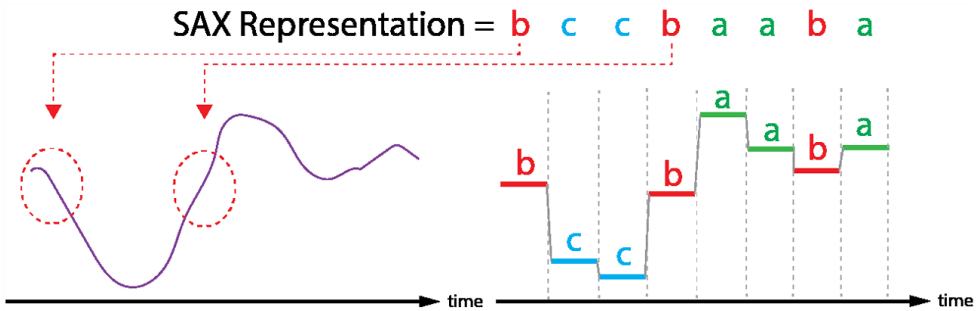
The Shape of Song

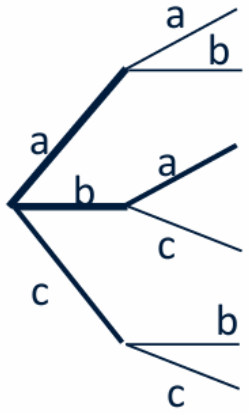
- Discard exact times, show **sequences** instead

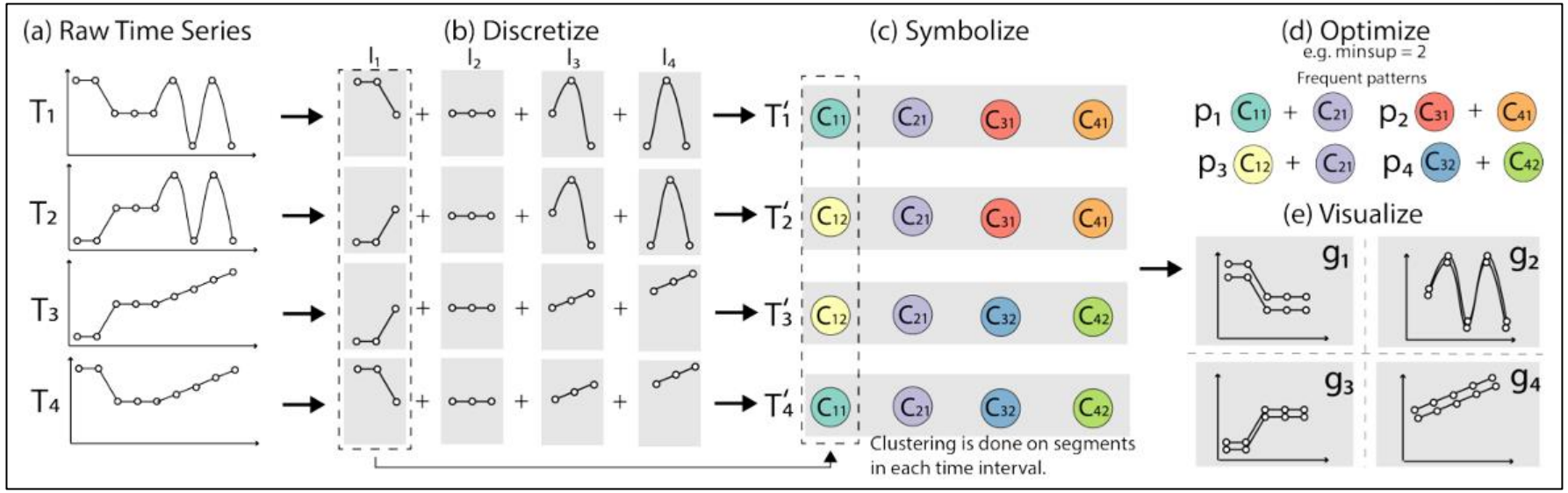
Sequence	Count
a	3
b	3
c	2

Sequence	Count
aa	1
ab	1
ac	0
ba	1
bb	0
bc	1
ca	0
cb	1
cc	1

- For example:
  - **Sequence Tree**
  - **Arc Diagram**

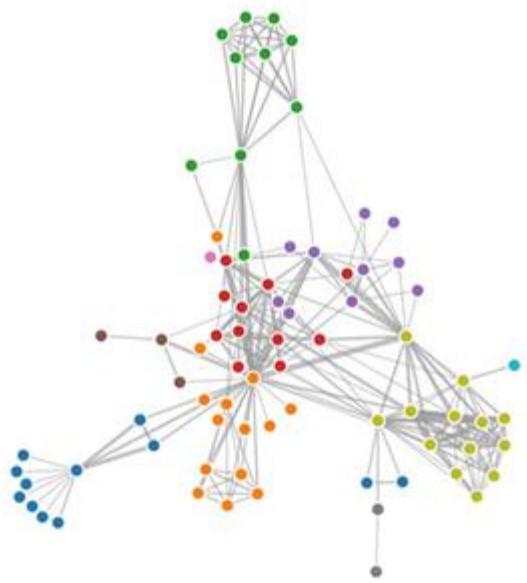






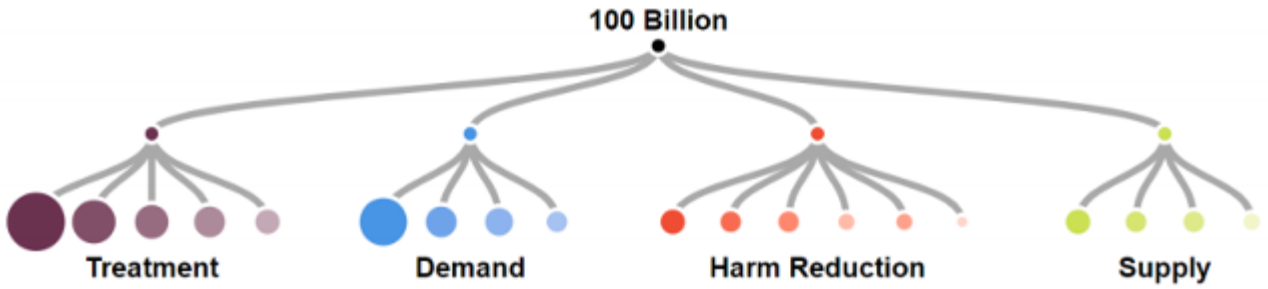
Graph Data

- Data on the edges:
  - Can be (bi)directional
  - Can have additional attributes
  - Can have multiple edges between the same node



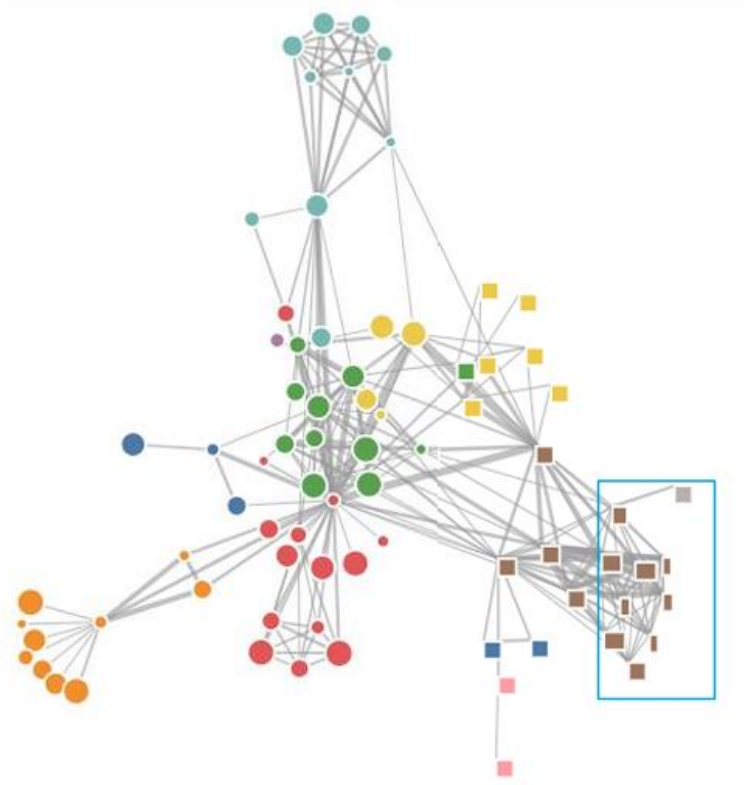
- **Tree**: a graph whose nodes are *partially ordered* (depth)

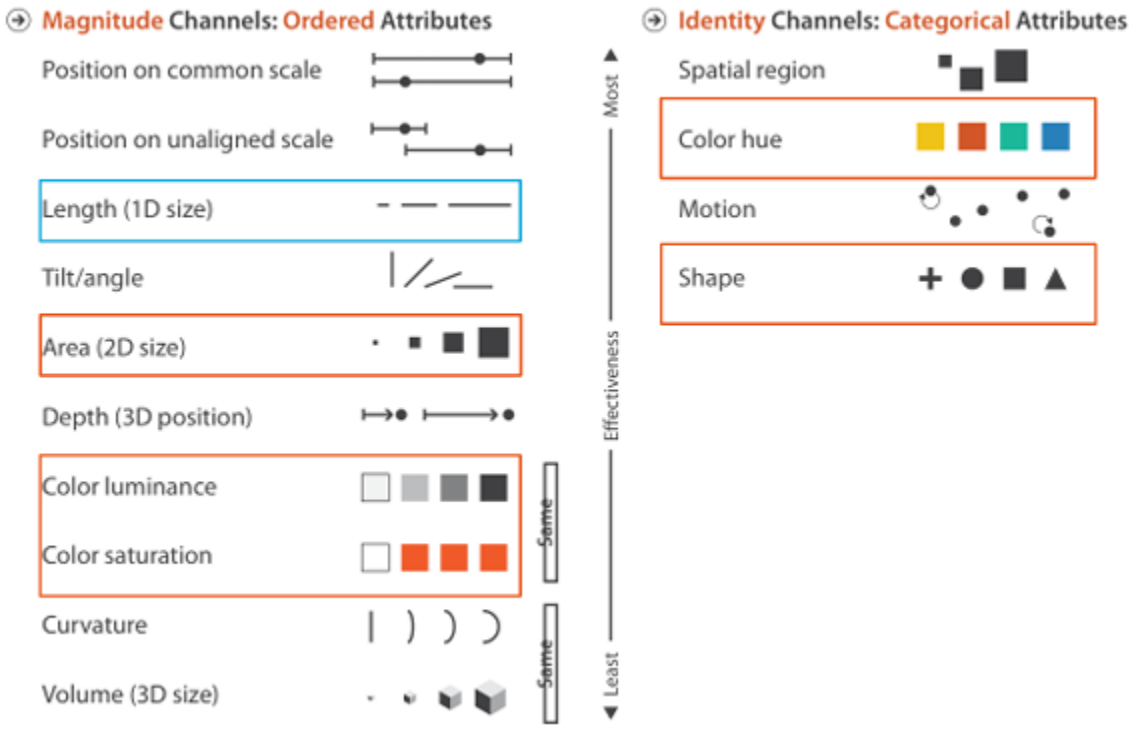
- How to visualize graph data:
  - **Node-Link Diagrams**
  - **Tree-Specific Visualizations**
  - **Alternate Graph Visualizations**



Node-Link Diagrams

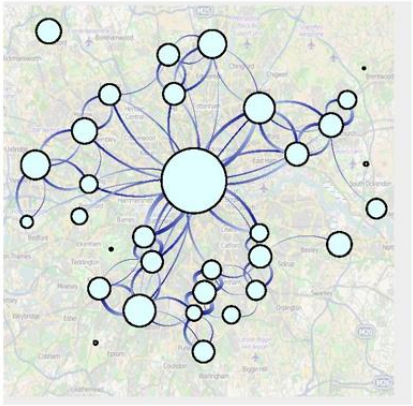
- **Nodes** can be encoded by multiple visual variables



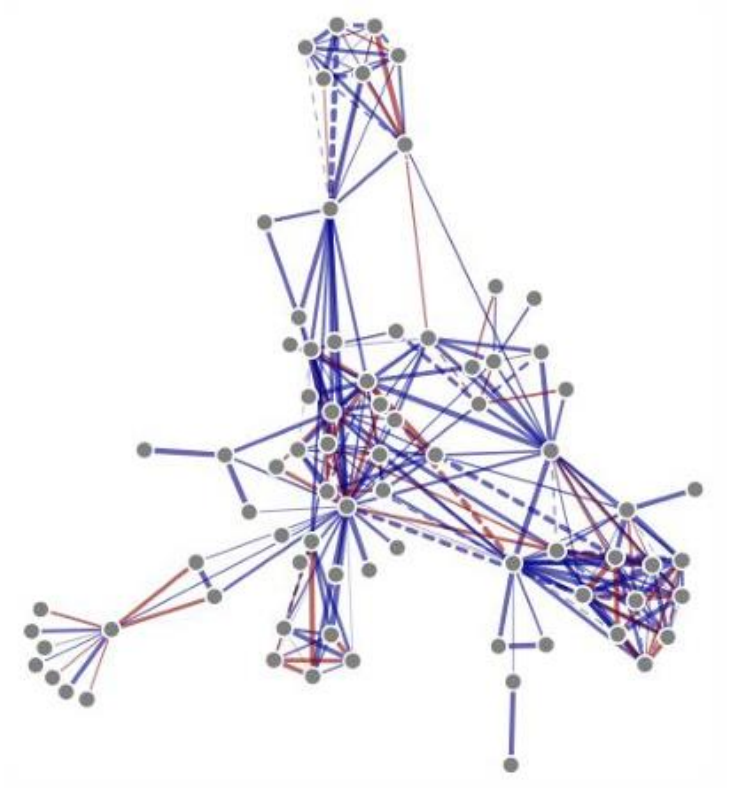


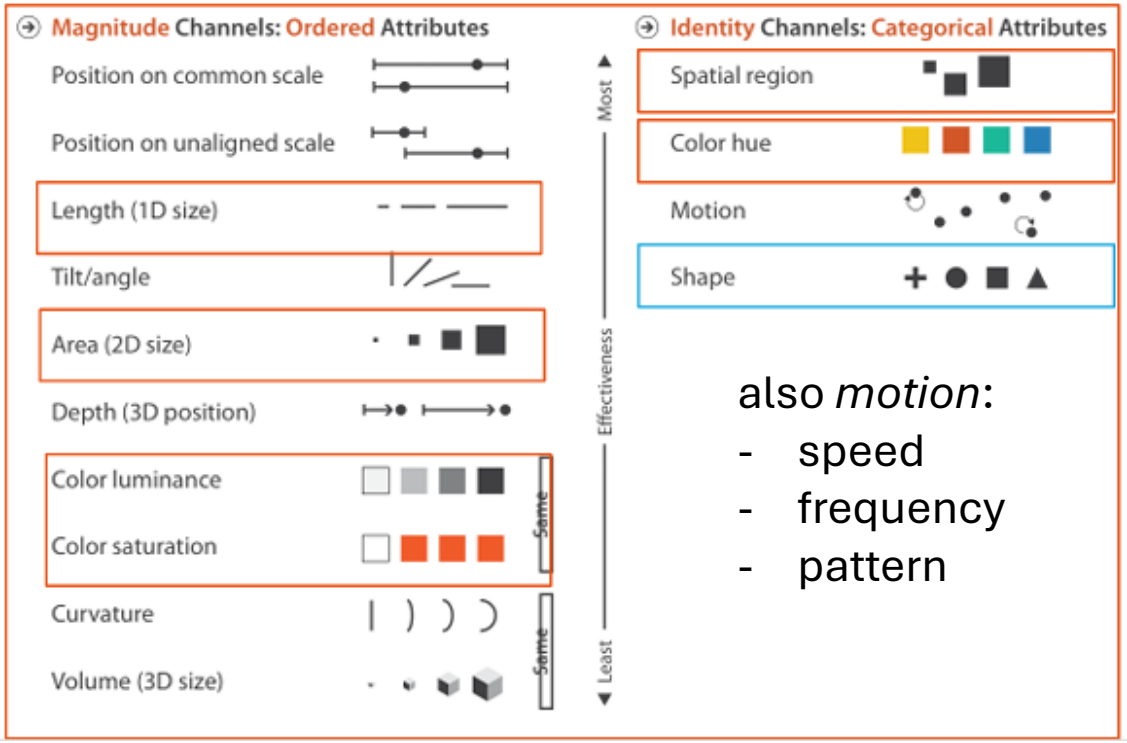
Node-Link Diagrams

- **Edges** can be encoded by multiple visual variables



edges encoded by curvature



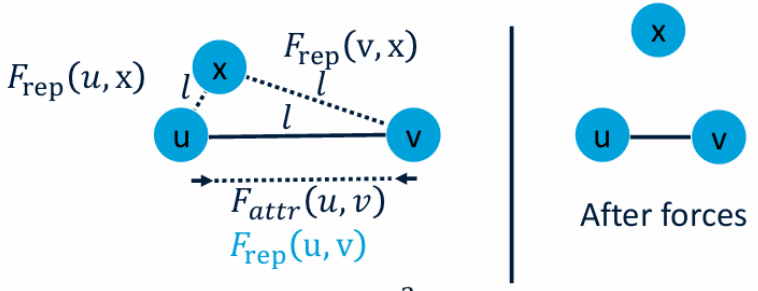


also *motion*:
- speed
- frequency
- pattern

Graph Layouts

- The **Layout** of a graph gives the positioning for each node
- Not all layouts are equal! Always ask: is it suitable for the data and task?
  - How many edge crossings are there?
  - How many bends in the edges are there?
  - What is the variance in the edge lengths?
- **Force-Directed Layout**: see the edges as springs pulling nodes together
  - **Attractive Force**: connected nodes pull each other in
  - **Repulsive Force**: nodes close to each other push away
  - Repeat calculations until stability

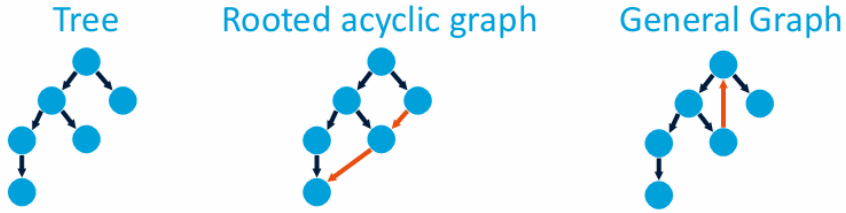
Formulas can also be adapted for:
- Minimal edge length
- Maximal edge length
- Consistent edge length



$$F_{attr}(u, v) = \frac{||p_u - p_v||^2}{l} \overrightarrow{p_v p_u}$$

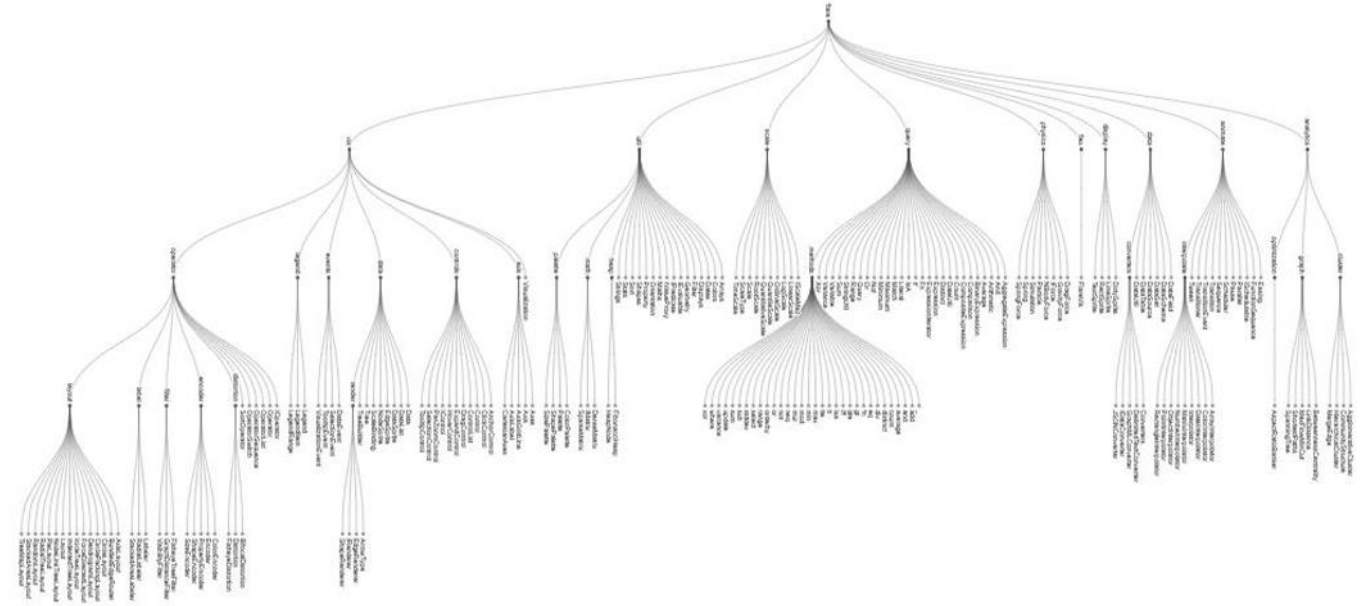
$$F_{\text{rep}}(u, v) = \frac{l^2}{||p_u - p_v||} \overrightarrow{p_v p_u}$$

Graph Layouts



- **Sugiyama** is a *layered* layout (*partially ordered*):

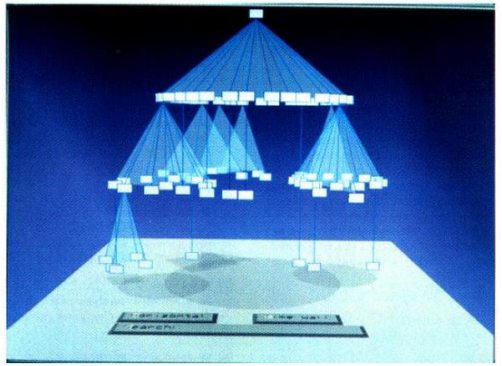
1. Determine the nodes for each layer
   - Determine a root node, then place nodes in layers by max. distance from the root
2. Minimize the number of edge crossings (in case of trees, optimal is 0)
3. Determine the final placement (center nodes over their children if possible)



Graph Layouts

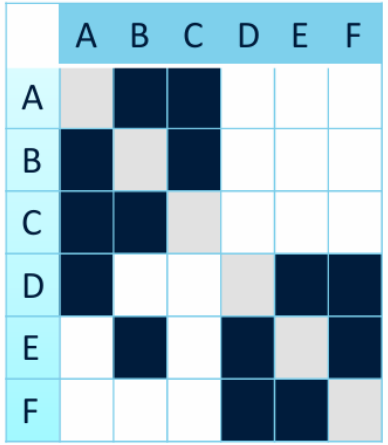
- Variations of layered layouts include:

- **Radial-Layered Layout**: position nodes using radial layers
  - Better data-ink ratio, but harder to see nodes at a specific layer

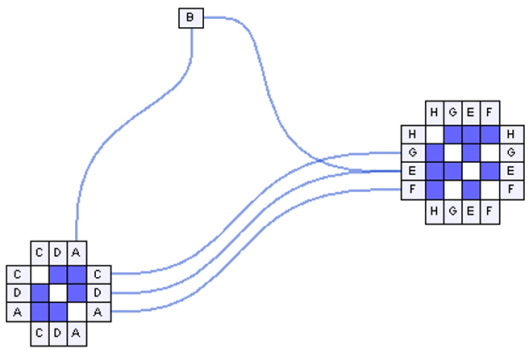


- **Cone Trees**: 3D variant of layered layouts
  - Requires interactivity to solve overlapping issues, usable if layers have spacial meaning

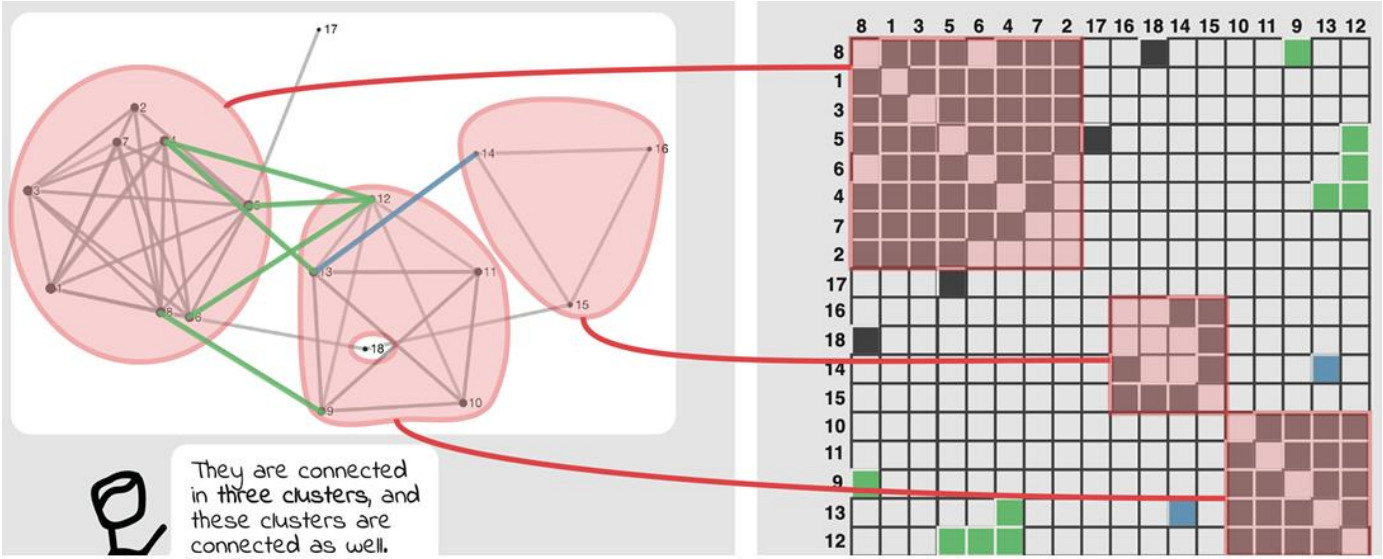
Alternate Graph Representations



- **Adjacency Matrix**: good for dense and clustered graphs
  - Try to avoid using text! Encode non-binary weights using colors
  - Order of columns / rows is *not* ordinal, can be reordered if needed (interactivity!)
  - Works well for looking up links, extremes, cliques, and clusters



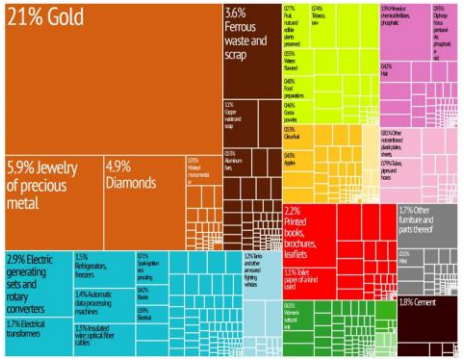
Hybrid Approach:
Node-Link + Adjacency Matrix

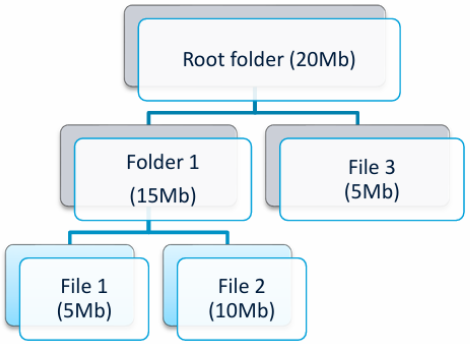


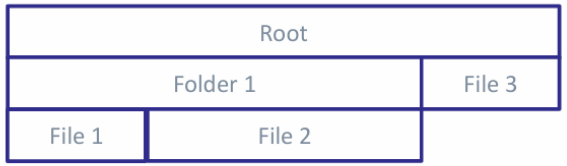
Clique: every node is connected to every other node

Cluster: *almost* every node is connected to every other node

Treemap

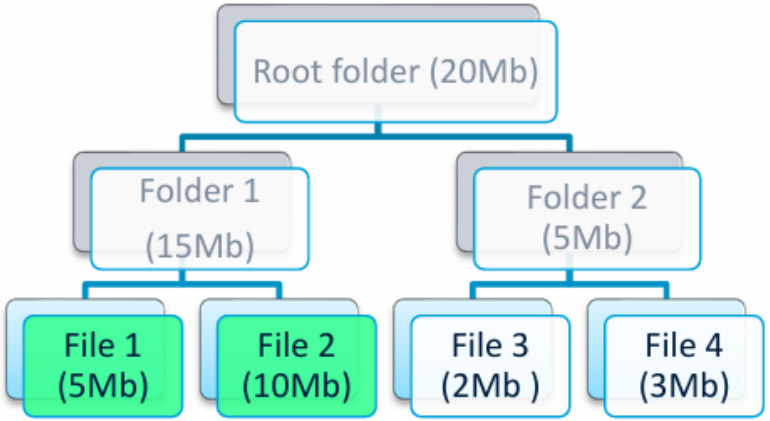


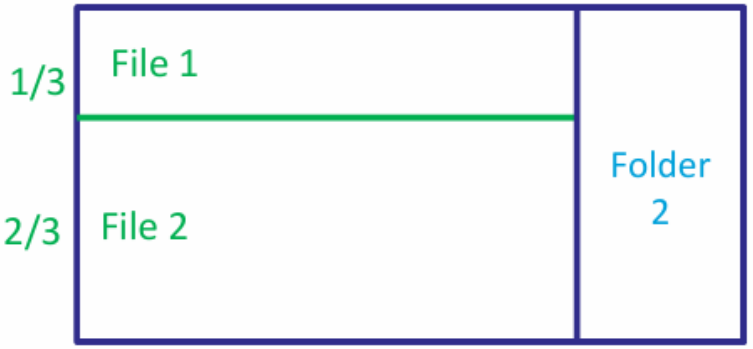


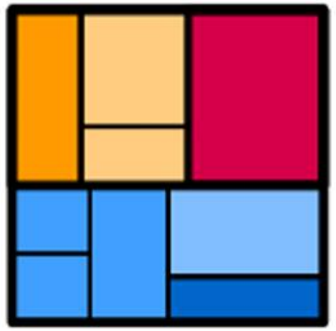


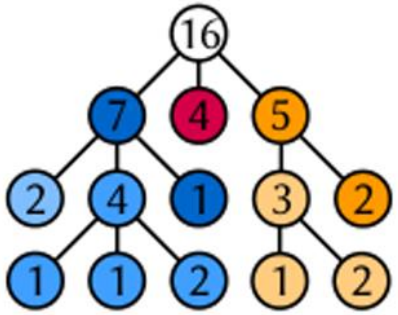
Icicle Plot: easier to see depth, but less space-efficient

- A **Treemap** shows tree data with quantitative values on the nodes
  - E.g. files on a hard drive. Typical tasks: exploring value distributions / locating large items
- **Slide-And-Dice** algorithm:
  1. Root initially takes the entire rectangle
  2. Split the rectangle into sub-rectangles according to the proportions of the children
  3. Repeat recursively, inverting cut direction each time









Treemap → Node-Link Diagram